

BobaLink Browser Extension

Developer Guide

Version: 1.0.0

Date: 2026-06-06

Classification: Internal

Table of Contents

1. Development Environment Setup

2. Project Structure

3. Building from Source

4. Testing

5. Code Style and Conventions

6. Architecture Decisions

7. Contributing

8. Extending the Extension

9. Debugging

1. Development Environment Setup

1.1 Prerequisites

Before you begin, ensure your development machine meets the following requirements:

Requirement	Minimum Version	Recommended	Purpose
Node.js	18 LTS	20 LTS	Runtime environment
npm	9.x	10.x	Package manager
Git	2.40	2.43+	Version control
A supported browser	Chrome 88+	Chrome 120+	Testing and development

Verify your environment:

```
node --version    # Should print v18.x.x or higher
npm --version     # Should print 9.x.x or higher
git --version     # Should print 2.40+ or higher
```

1.2 Cloning the Repository

```
# Clone the repository
git clone https://github.com/bobaproject/bobalink.git
```

1.3 Installing Dependencies

Key dependencies that will be installed:

Package	Version	Purpose
wxt	^0.17	Extension build framework
typescript	^5.0	Type safety
vite	^5.0	Module bundler (via WXT)
jest	^29.0	Unit testing framework
@playwright/test	^1.40	E2E testing framework
eslint	^8.0	Code linting
prettier	^3.0	Code formatting
bencode-js	^3.0	Bencode parsing for queries
lz-string	^1.5	Compression for queries
webextension-polyfill	^0.10	Firefox compatibility

1.4 IDE Recommendations

Visual Studio Code (Recommended)

```

{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "explicit"
  },
  "typescript.tsdk": "node_modules/typescript/lib",
  "typescript.preferences.importModuleSpecifier": "relative",
  "files.exclude": {
    "**/node_modules": true,
    "**/.output": true,
    "**/dist": true
  },
  "search.exclude": {
    "**/node_modules": true,
    "**/dist": true,
    "**/.output": true,
    "**/*.lock": true
  }
}

```

.vscode/extensions.json :

```

{
  "recommendations": [
    "esbenp.prettier-vscode",
    "dbaeumer.vscode-eslint",
    "ms-vscode.vscode-typescript-next",
    "bradlc.vscode-tailwindcss",
    "ryanluker.vscode-coverage-gutters",
    "orta.vscode-jest"
  ]
}

```

Extension	Publisher
Prettier	esbenp.prettier-vscode
ESLint	dbaeumer.vscode-eslint
Tailwind CSS IntelliSense	bradlc.vscode-tailwindcss
Coverage Gutters	ryanluker.vscode-coverage-gutters
Jest	orta.vscode-jest

WebStorm / IntelliJ IDEA

If using JetBrains IDEs:

1. Enable the Prettier plugin: **Settings** → **Languages & Frameworks** → **JavaScript** → **Prettier**.
2. Enable ESLint: **Settings** → **Languages & Frameworks** → **JavaScript** → **ESLint**.
3. Set TypeScript version: **Settings** → **Languages & Frameworks** → **TypeScript** → `node_modules/typescript/lib`.

2. Project Structure

2.1 Directory Layout

```
bobalink/
├─ src/
│   ├── assets/                    # Static assets (icons, images)
│   │   ├── icon-16.png
│   │   ├── icon-32.png
│   │   ├── icon-48.png
│   │   └─ icon-128.png
│   ├── background/                # Service worker (MV3 background)
│   │   ├── index.ts              # Entry point
│   │   ├── message-router.ts
│   │   └─ init.ts
│   ├── content-scripts/           # Scripts injected into web page
│   │   ├── index.ts              # Entry point
│   │   ├── magnet-scanner.ts
│   │   ├── dom-observer.ts
│   │   └─ torrent-parser.ts
│   ├── popup/                     # Toolbar popup UI
│   │   ├── index.html
│   │   ├── index.ts
│   │   ├── popup.css
│   │   └─ components/
│   │       ├── torrent-list.ts
│   │       ├── queue-view.ts
│   │       ├── status-bar.ts
│   │       └─ tab-groups.ts
│   ├── options/                   # Options/settings page
│   │   ├── index.html
│   │   ├── index.ts
│   │   ├── options.css
│   │   └─ components/
│   │       ├── server-config.ts
│   │       ├── download-prefs.ts
│   │       ├── queue-settings.ts
│   │       └─ ui-settings.ts
│   ├── modules/                   # Shared business logic modules
│   │   └─ api-client.ts
```


2.2 Module Descriptions

Background (Service Worker)

Module	Responsibility
<code>index.ts</code>	Service worker entry point; registers event listeners
<code>message-router.ts</code>	Routes incoming extension messages to appropriate handlers
<code>init.ts</code>	Extension initialization: config loading, health monitoring

Content Scripts

Module	Responsibility
<code>index.ts</code>	Content script entry; initializes scanner and observer
<code>magnet-scanner.ts</code>	Scans DOM for magnet links and <code>.torrent</code> files
<code>dom-observer.ts</code>	<code>MutationObserver</code> wrapper for dynamic content
<code>torrent-parser.ts</code>	Parses magnet URIs and <code>.torrent</code> file metadata

Modules (Shared Logic)

Module	Responsibility
<code>api-client.ts</code>	HTTP client for Boba and qBitTorrent
<code>auth-manager.ts</code>	Authentication: login, token refresh, s
<code>badge-manager.ts</code>	Toolbar badge state: color, text, anima
<code>config-manager.ts</code>	Configuration CRUD with validation
<code>crypto-utils.ts</code>	AES-256-GCM encryption, PBKDF2
<code>discovery-service.ts</code>	mDNS-like local network server disco
<code>health-monitor.ts</code>	Periodic health checks; connection sta
<code>i18n.ts</code>	Internationalization: message lookup,
<code>notification-manager.ts</code>	Browser notification creation and man
<code>queue-manager.ts</code>	Offline queue: enqueue, dequeue, retr
<code>rate-limiter.ts</code>	Token bucket rate limiter for API call
<code>state-manager.ts</code>	Central state store; reactive updates to
<code>tab-manager.ts</code>	Tab group enumeration; batch scan on

3. Building from Source

3.1 Development Build

Start the development server with hot module replacement:

```
npm run dev
```

This command:

1. Starts WXT in development mode.
2. Watches source files for changes and rebuilds automatically.
3. Outputs to `.output/` directory.
4. Provides a Chrome-compatible build by default.

Output structure:

```
.output/  
├─ chrome-mv3-dev/      # Chrome development build  
│   ├── manifest.json  
│   ├── background.js  
│   ├── content-scripts/  
│   ├── popup/  
│   └─ options/
```

Loading the development build:

3.2 Production Build

Create optimized production builds for all targets:

```
npm run build
```

This generates:

```
.output/  
├─ chrome-mv3-prod/      # Chrome production build  
├─ firefox-mv3-prod/     # Firefox production build  
├─ opera-mv3-prod/       # Opera production build  
└─ edge-mv3-prod/        # Edge production build (same as Chrome)
```

Each build is:

- Minified and tree-shaken.
- Source-mapped (`.map` files).
- Ready for packaging as ZIP for store submission.

3.3 Cross-Browser Builds

Build for specific browsers:

```
# Chrome only  
npx wxt build -b chrome  
  
# Firefox only  
npx wxt build -b firefox
```

```
import { defineConfig } from 'wxt';

export default defineConfig({
  srcDir: 'src',
  outDir: '.output',
  modules: ['@wxt-dev/module-typescript'],
  manifest: {
    name: '__MSG_extName__',
    description: '__MSG_extDescription__',
    version: '1.0.0',
    permissions: ['activeTab', 'storage', 'contextMenus', 'no'],
    host_permissions: ['https://*/'],
    action: {
      default_popup: 'popup/index.html'
    },
    options_ui: {
      page: 'options/index.html',
      open_in_tab: true
    },
    commands: {
      'send-current-page': {
        suggested_key: { default: 'Ctrl+Shift+B', mac: 'Command+Shift+B' },
        description: 'Send all torrents on current page'
      }
    }
  }
});
```

4. Testing

```
# Run all tests
npx jest

# Run with coverage
npx jest --coverage

# Run in watch mode (during development)
npx jest --watch

# Run a specific test file
npx jest tests/unit/utils/magnet-uri.test.ts

# Run tests matching a pattern
npx jest --testNamePattern="parseMagnetUri"
```

Coverage reporting:

Coverage reports are generated in `coverage/`:

```
coverage/
├── lcov-report/      # HTML report (open index.html in browser)
├── lcov.info         # LCOV format for CI integration
└── coverage-summary.json
```

Coverage thresholds (enforced in CI):

Metric	Minimum
Statements	80%
Branches	75%

```
# Install Playwright browsers (one-time setup)
npx playwright install chromium firefox

# Run all E2E tests
npm run test:e2e

# Run in headed mode (visible browser)
npx playwright test --headed

# Run a specific spec
npx playwright test tests/e2e/specs/detection.spec.ts

# Run with UI mode (interactive debugging)
npx playwright test --ui

# Run on specific browser
npx playwright test --project=chromium
n``

**E2E Test Structure:**
```

tests/e2e/

```
├── fixtures/ # Test data
│   ├── sample.torrent
│   ├── magnet-links.html
│   └── mock-server.ts
├── specs/ # Test specifications
│   ├── detection.spec.ts # Torrent detection tests
│   ├── sending.spec.ts # Send-to-server tests
│   ├── queue.spec.ts # Queue management tests
│   └── auth.spec.ts # Authentication tests
```

4.3 Writing New Tests

Unit Test Template

```
```typescript
import { parseMagnetUri } from '@src/utils/magnet-uri';

describe('parseMagnetUri', () => {
 it('should parse a complete magnet URI', () => {
 const uri = 'magnet:?xt=urn:btih:abc123def456&dn=Test+File';
 const result = parseMagnetUri(uri);

 expect(result.infohash).toBe('abc123def456');
 expect(result.displayName).toBe('Test File');
 expect(result.trackers).toContain('https://tracker.example.com');
 });

 it('should throw on invalid magnet URI', () => {
 expect(() => parseMagnetUri('not-a-magnet')).toThrow('Invalid magnet URI');
 });

 it('should handle missing display name', () => {
 const uri = 'magnet:?xt=urn:btih:abc123def456';
 const result = parseMagnetUri(uri);
 expect(result.displayName).toBe('Unknown Torrent');
 });
});
```



## E2E Test Template

```
import { test, expect } from '@playwright/test';
import { PopupPage } from '../page-objects/popup.page';

test.describe('Torrent Detection', () => {
 test('should detect magnet links on a page', async ({ page, extensionId }) => {
 // Arrange: Load a test page with magnet links
 await page.goto(`file://${__dirname}/../fixtures/magnet-1.html`);

 // Act: Open the extension popup
 const popup = new PopupPage(page, extensionId);
 await popup.open();

 // Assert: Verify torrents are listed
 const torrentCount = await popup.getTorrentCount();
 expect(torrentCount).toBe(3);

 const firstTorrentName = await popup.getTorrentName(0);
 expect(firstTorrentName).toContain('Ubuntu');
 });
});
```

## 4.4 Test Coverage Reports

Generate and view coverage:

```
npm run test:coverage
```

The HTML report opens automatically. Key files to monitor:

## 5. Code Style and Conventions

---

### 5.1 TypeScript Strict Mode

The project uses TypeScript strict mode. Key rules:

```
{
 "compilerOptions": {
 "strict": true,
 "noImplicitAny": true,
 "strictNullChecks": true,
 "strictFunctionTypes": true,
 "strictBindCallApply": true,
 "strictPropertyInitialization": true,
 "noImplicitReturns": true,
 "noFallthroughCasesInSwitch": true,
 "noUncheckedIndexedAccess": true
 }
}
```

**No `any` types in production code.** Use `unknown` with type guards:

```
// Bad
function process(data: any): any {
 return data.value;
}

// Good
function process(data: unknown): string {
 if (typeof data === 'object' && data !== null && 'value' in data) {
 return data.value;
 }
 return '';
}
```

## 5.2 ESLint Configuration

```
// eslint.config.js
export default [
 {
 files: ['src/**/*.ts'],
 languageOptions: {
 parser: '@typescript-eslint/parser',
 parserOptions: {
 project: './tsconfig.json'
 }
 },
 },
 {
 plugins: {
 '@typescript-eslint': typescriptEslint
 },
 rules: {
 '@typescript-eslint/no-explicit-any': 'error',
 '@typescript-eslint/no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
 '@typescript-eslint/explicit-function-return-type': 'warn',
 '@typescript-eslint/no-non-null-assertion': 'error',
 '@typescript-eslint/prefer-nullish-coalescing': 'error',
 '@typescript-eslint/prefer-optional-chain': 'error',
 'no-console': ['warn', { allow: ['error', 'warn'] }]
 }
 }
];
```

## 5.3 Prettier Configuration

```
// prettier.config.js
export default {
 semi: true
```

## 5.4 Naming Conventions

Construct	Convention	Example
Variables / Functions	camelCase	<code>sendTorrent</code> , <code>getSt</code>
Constants	UPPER_SNAKE_CASE	<code>MAX_QUEUE_SIZE</code> , <code>DE</code>
Types / Interfaces	PascalCase	<code>TorrentInfo</code> , <code>Serve</code>
Classes	PascalCase	<code>QueueManager</code> , <code>ApiC</code>
Enums	PascalCase + PascalCase members	<code>enum ConnectionState</code> <code>Disconnected }</code>
Files	kebab-case	<code>magnet-scanner.ts</code>
Test files	<code>*.test.ts</code> or <code>*.spec.ts</code>	<code>magnet-uri.test.ts</code>
Private members	Leading underscore	<code>_internalQueue</code> , <code>_t</code>

## 5.5 Comment Style (JSDoc)

All public APIs must include JSDoc comments:

```
/**
 * Parses a magnet URI string into a structured TorrentInfo o
 *
```

Comment guidelines:

- Explain **why**, not **what** (the code shows what).
- Document all parameters, return values, and thrown errors.
- Include usage examples for complex functions.
- Use `TODO(username): description` for known technical debt.
- Use `FIXME(username): description` for bugs that need fixing.

## 6. Architecture Decisions

### 6.1 Why MV3 Over MV2

**Decision:** Use Manifest V3 (MV3) instead of Manifest V2 (MV2).

**Rationale:**

Factor	MV2	MV3
Chrome Web Store acceptance	Being deprecated	Required
Service worker vs background page	Persistent background page	Ephemera
Performance	Background page always running	Service w events
Memory usage	Higher (persistent)	Lower (ep

## 6.2 Why WXT Over Raw Webpack/Vite

**Decision:** Use WXT as the build framework instead of configuring Webpack

**Rationale:**

Factor	Raw Vite	WXT
Extension-specific scaffolding	None	Built-in
HMR for content scripts	Manual config	Automatic
Multi-browser builds	Manual config	Automatic
Manifest generation	Manual	Auto-generated from code
File-based routing	None	Entry points auto-discovered
Dev server with auto-reload	Manual config	Built-in
Bundle size	Comparable	Comparable (thin wrappers)
Community size	Large	Growing

**WXT specifically solves:**

- Automatic discovery of `src/popup/` , `src/options/` , `src/background` entries.
- Browser-specific build variants (Chrome MV3, Firefox MV3 with polyfills)
- Dev mode that auto-reloads the extension on file change.
- Automatic i18n message placeholder substitution in the manifest.

Permission Model	Install Prompt	Privacy
<code>host_permissions: &lt;all_urls&gt;</code>	Scary "read all website data"	Poor
<code>activeTab</code>	No prompt (on-demand)	Excellent

**Implementation:**

- Primary detection uses `activeTab` + user gesture (clicking the extension icon)
- Optional `host_permissions` for advanced users who want proactive detection
- The `optional_permissions` field in the manifest allows users to grant access

## 6.4 Why AES-256-GCM for Encryption

**Decision:** Use AES-256-GCM via the Web Crypto API for credential encryption

**Rationale:**

Algorithm	Mode	Authentication	Performance
AES-128-CBC	No	No (requires HMAC)	Fast
AES-256-CBC	No	No (requires HMAC)	Fast
AES-256-GCM	Yes	Built-in	Fast
ChaCha20-Poly1305	Yes	Built-in	Fast

AES-256-GCM was chosen because:

1. It provides authenticated encryption (confidentiality + integrity) in a single

# 7. Contributing

## 7.1 Git Workflow (GitHub Flow)

BobaLink follows the GitHub Flow branching model:



Branch naming conventions:

Prefix	Purpose	Example
feature/	New features	feature/tab-cards
bugfix/	Bug fixes	bugfix/badge-click
hotfix/	Critical production fixes	hotfix/auth-refresh
docs/	Documentation updates	docs/api-examples
refactor/	Code refactoring	refactor/queue-consumer
test/	Test additions/improvements	test/queue-consumer

## 7.2 Commit Message Conventions (Conventional



Type	Use When	Example
feat	Adding a new feature	feat(popup): add tab
fix	Fixing a bug	fix(queue): resolve r
docs	Documentation changes	docs(api): add qBitTo
style	Code formatting (no logic change)	style(lint): fix trai
refactor	Code restructuring	refactor(auth): extra
test	Adding or updating tests	test(queue): add retr
chore	Build/tooling changes	chore(deps): update w
perf	Performance improvements	perf(scanner): deboun
security	Security-related changes	security(crypto): inc iterations

## Scopes:

Scope	Description
popup	Popup UI
options	Options page
content	Content scripts
background	Service worker
api	API client
queue	Queue manager
auth	Authentication
crypto	Encryption utilities
i18n	Internationalization
build	Build system
deps	Dependencies

**Breaking changes:**

Append `!` after the type/scope and include `BREAKING CHANGE:` in the foot

```
feat(api)!: remove v1 auth endpoint

BREAKING CHANGE: The /api/v1/auth/login endpoint has been removed.
Use /api/v2/auth/login instead.
```

## ## Description

Brief description of the change and its purpose.

## ## Type of Change

- ☐ Bug fix
- ☐ New feature
- ☐ Breaking change
- ☐ Documentation update
- ☐ Refactoring

## ## Testing

- ☐ Unit tests added/updated
- ☐ E2E tests added/updated
- ☐ Manual testing performed

## ## Checklist

- ☐ Code follows project style guidelines
- ☐ Self-review completed
- ☐ JSDoc comments added for public APIs
- ☐ No `any` types introduced
- ☐ TypeScript compiles without errors
- ☐ ESLint passes without warnings
- ☐ Test coverage meets thresholds

## ## Related Issues

Fixes #123

Relates to #456

## 7.4 Code Review Checklist

Reviewers should verify:

Check	Description
Functionality	Does the change do what it claims?
Tests	Are there adequate tests? Do they pass?
Edge cases	Are error paths and boundary conditions handled?
Security	Does the change introduce any security risks?
Performance	Could this cause performance issues at scale?
i18n	Are all user-facing strings externalized?
Accessibility	Are ARIA labels and keyboard navigation present?
Type safety	Are there no <code>any</code> types or unchecked nulls?
Documentation	Is JSDoc updated? Is the user guide updated if needed?

## 7.5 Release Process



**Version numbering** follows Semantic Versioning (SemVer):

Bump	When
Major (X.0.0)	Breaking API changes

## 8.1 Adding New Torrent Site Selectors

Some torrent sites use non-standard HTML structures. You can add custom selectors to support these sites.

1. Create a new file in `src/content-scripts/selectors/`:

```
// src/content-scripts/selectors/nyaa-si.ts
import { SiteSelector } from '@src/types/selector';

export const nyaaSiSelector: SiteSelector = {
 // Match against hostname
 hostname: /nyaa\.si$/,

 // CSS selector for magnet links
 magnetSelector: 'a[href^="magnet:"]',

 // CSS selector for .torrent file links
 torrentSelector: 'a[href$=".torrent"]',

 // Optional: Extract display name from a different element
 nameExtractor: (linkElement: HTMLAnchorElement): string | null => {
 const row = linkElement.closest('tr');
 if (row) {
 const titleCell = row.querySelector('td:nth-child(2) a');
 return titleCell?.textContent?.trim() || null;
 }
 return null;
 },

 // Optional: Extract size from page
 sizeExtractor: (linkElement: HTMLAnchorElement): number | null => {
 const row = linkElement.closest('tr');
 if (row) {
 const sizeCell = row.querySelector('td:nth-child(4)');
 return parseSizeString(sizeCell?.textContent || '');
 }
 return null;
 }
};
```

```
import { nyaaSiSelector } from './nyaa-si';

export const siteSelectors: SiteSelector[] = [
 // ... existing selectors
 nyaaSiSelector,
];
```

### 3. Write a test:

```
// tests/unit/content-scripts/selectors/nyaa-si.test.ts
import { nyaaSiSelector } from '@src/content-scripts/selector

describe('nyaaSiSelector', () => {
 it('should match nyaa.si hostname', () => {
 expect(nyaaSiSelector.hostname.test('nyaa.si')).toBe(true)
 expect(nyaaSiSelector.hostname.test('sukebei.nyaa.si')).t
 });

 it('should extract name from table row', () => {
 document.body.innerHTML = `
 <table>
 <tr>
 <td></td>
 <td><a>My Torrent Title</td>
 <td></td>
 <td>1.2 GiB</td>
 <td>Magnet
 </tr>
 </table>
 `;
 const link = document.querySelector('a[href^="magnet"]')
 expect(nyaaSiSelector.nameExtractor?.(link)).toBe('My Tor
 });
});
```

```

// src/modules/auth/oauth2-handler.ts
import { AuthHandler, AuthCredentials } from '@src/types/auth'

export class OAuth2Handler implements AuthHandler {
 private token: string | null = null;
 private refreshToken: string | null = null;
 private expiresAt = 0;

 async authenticate(credentials: AuthCredentials): Promise<v
 const response = await fetch(`${credentials.serverUrl}/oa
 method: 'POST',
 headers: { 'Content-Type': 'application/json' },
 body: JSON.stringify({
 grant_type: 'password',
 client_id: credentials.clientId,
 username: credentials.username,
 password: credentials.password,
 }),
 });

 if (!response.ok) {
 throw new AuthError('OAuth2 authentication failed', { c
 }

 const data = await response.json();
 this.token = data.access_token;
 this.refreshToken = data.refresh_token;
 this.expiresAt = Date.now() + data.expires_in * 1000;
 }

 async getAuthHeaders(): Promise<Record<string, string>> {
 if (!this.token || Date.now() >= this.expiresAt) {
 await this.refresh();
 }
 return { Authorization: `Bearer ${this.token}` };
 }
}

```

```
// src/modules/auth-manager.ts
import { OAuth2Handler } from '../auth/oauth2-handler';

const authHandlers: Record<AuthMethod, () => AuthHandler> = {
 'cookie': () => new CookieHandler(),
 'api-key': () => new ApiKeyHandler(),
 'basic': () => new BasicAuthHandler(),
 'custom-header': () => new CustomHeaderHandler(),
 'oauth2': () => new OAuth2Handler(), // Add this line
};
```

4. Update the Options UI to include OAuth2 configuration fields.

## 8.3 Adding New Keyboard Shortcuts

To add a new keyboard shortcut:

1. Define the command in the manifest:

```
{
 "commands": {
 "send-current-page": {
 "suggested_key": { "default": "Ctrl+Shift+B" },
 "description": "Send all torrents on current page"
 },
 "your-new-command": {
 "suggested_key": { "default": "Ctrl+Shift+N" },
 "description": "Description of what it does"
 }
 }
}
```



### 3. Implement the handler:

```
// src/background/command-handlers.ts
export async function handleYourNewCommand(tab?: chrome.tabs.Tab) {
 if (!tab?.id) return;

 // Your implementation here
 const result = await chrome.tabs.sendMessage(tab.id, {
 type: 'YOUR_NEW_ACTION'
 });

 // Update badge or show notification
 await badgeManager.setStatus('processing');
}
```

### 4. Add i18n strings:

```
{
 "command_your_new_command": {
 "message": "Your new command description"
 }
}
```

---

## 9. Debugging

### 9.1 Chrome DevTools for Extensions

Service Workers

### Popup:

1. Click the BobaLink icon to open the popup.
2. Right-click inside the popup → **"Inspect"**.
3. This opens a dedicated DevTools window for the popup.

### Extension ID:

Mode	Extension ID Location
Development	<code>chrome://extensions</code> → BobaLink → ID field
Production	Fixed ID from Chrome Web Store

## 9.2 Firefox about:debugging

1. Navigate to `about:debugging`.
2. Click **"This Firefox"**.
3. Find BobaLink under Extensions.
4. Click **"Inspect"** to open DevTools.
5. Use Browser Console ( `Ctrl+Shift+J` ) for all extension logs.

## 9.3 Viewing Service Worker Logs

The service worker logs extensively for debugging. Log levels:

```
// src/utils/logger.ts
enum LogLevel {
 DEBUG = 0, // Verbose internal state
```

Log output examples:

```
[BobaLink] [INFO] Service worker initialized, version 1.0.0
[BobaLink] [DEBUG] Health check: GET https://boba.local:8443/
[BobaLink] [INFO] Health check passed: Boba v1.4.2, latency
[BobaLink] [DEBUG] Magnet detected: abc123... on https://exam
[BobaLink] [INFO] Torrent sent: abc123... (Ubuntu 24.04)
[BobaLink] [WARN] Send failed for def456... (Network timeout)
[BobaLink] [ERROR] Auth failed: invalid API key
```

9.4 Inspecting Storage

Extension storage can be inspected via DevTools:

Storage Type	Chrome
chrome.storage.local	DevTools → Application → Storage → Extension Storage
chrome.storage.session	DevTools → Application → Storage → Extension Storage (Session)

Viewing storage via console:

```
// View all stored data
chrome.storage.local.get(null, (data) => console.log(data));

// View configuration
chrome.storage.local.get('config', ({ config }) => console.log(config));
```

### Diagnostic steps:

1. Check DevTools → Console for injection errors.
2. Verify the content script is registered in `manifest.json`.
3. Check if the page has a restrictive CSP:

```
javascript // In page console document.querySelector('meta[Security-Policy]')?.content
```

4. Check the service worker logs for injection errors.

## Scenario: API Calls Failing

**Symptoms:** "Send failed" notifications, queue filling up.

### Diagnostic steps:

1. Open service worker DevTools → Network tab.
2. Trigger a send and watch the request.
3. Check request headers (especially `Referer` for qBitTorrent).
4. Check response status and body.
5. Verify CORS headers if cross-origin.

## Scenario: Queue Not Processing

**Symptoms:** Items stuck in queue, no retry attempts.

### Diagnostic steps:

1. Check service worker is alive: `chrome://extensions` → Service Worker
2. Check for `chrome.alarms` registration:

```
javascript chrome.alarms.getAll((alarms) => console.log(ala
```

3. Check queue processing logs in service worker console.
4. Manually trigger processing:

```
javascript chrome.runtime.sendMessage({ type: 'PROCESS_QUEUE'
```

*End of Developer Guide*